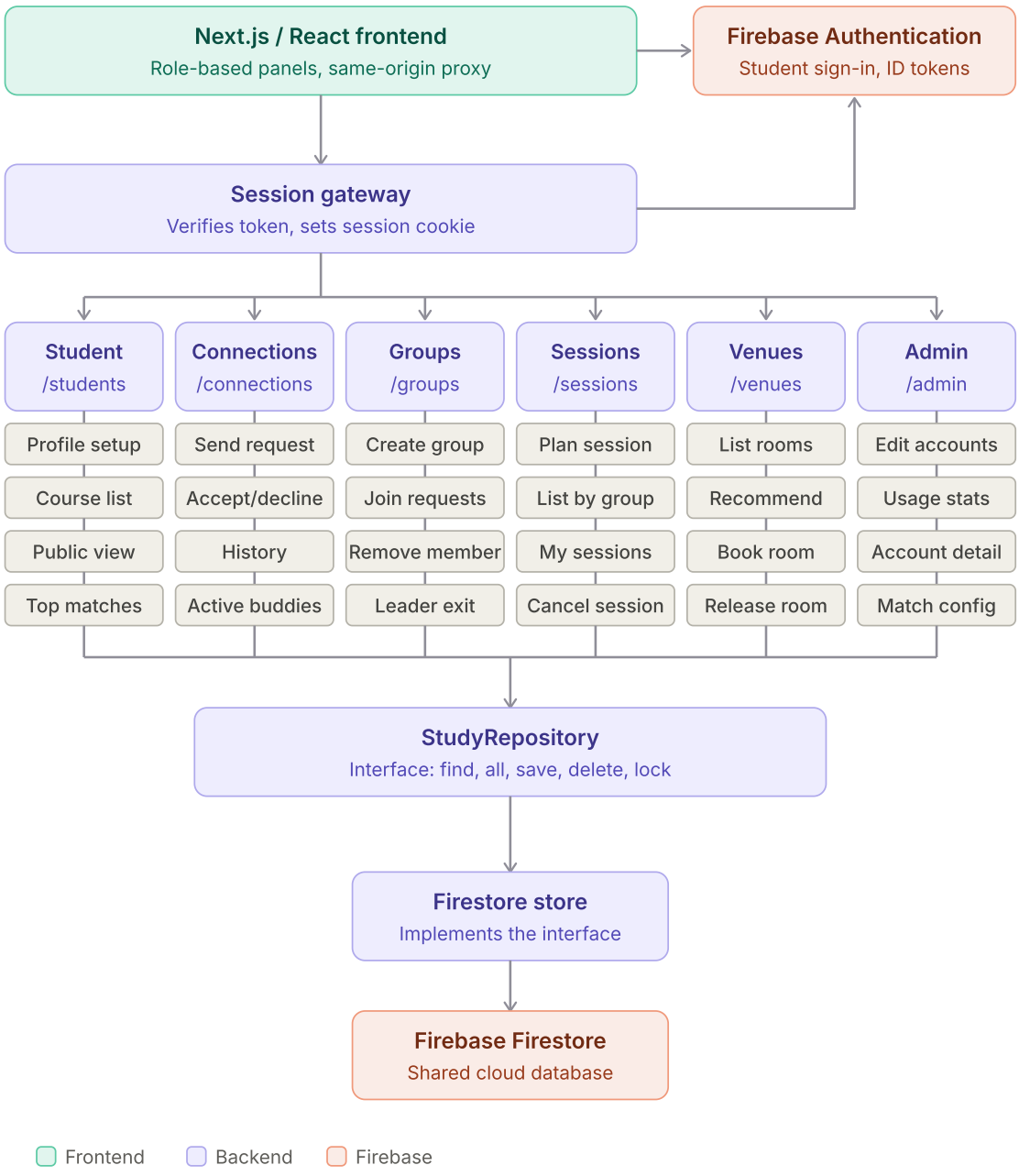
Study Buddy Matcher — Architecture Reference

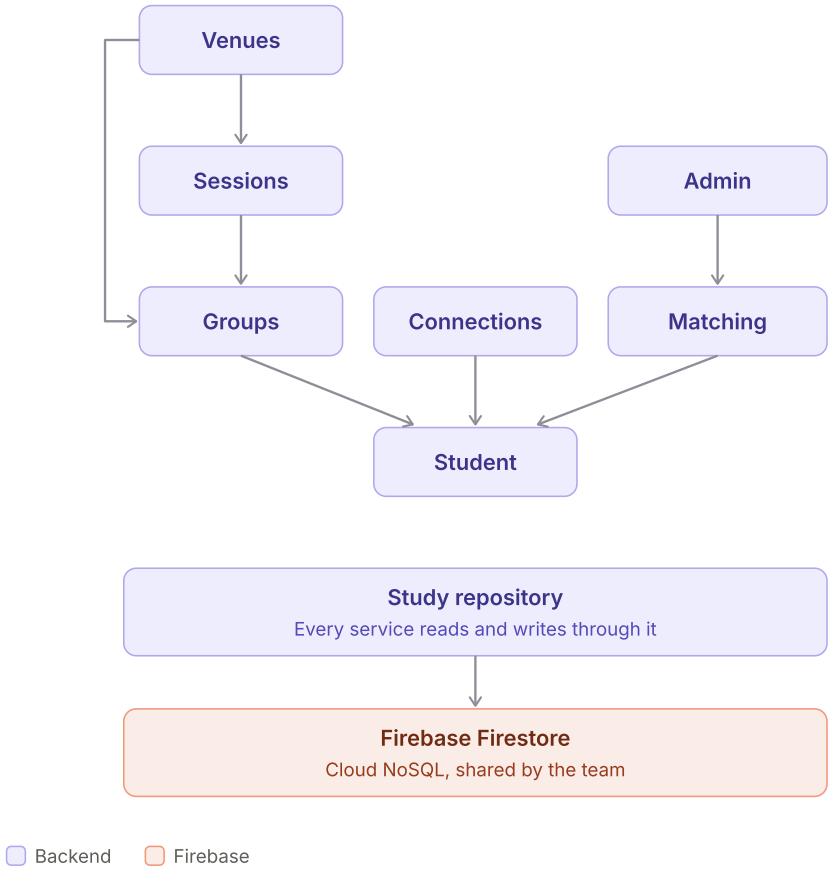
System diagrams, module/API reference, and process flows, compiled from the project chat.

1. System architecture



Students sign in with Firebase Auth; the gateway verifies the token. Six modules sit behind it. Every service persists through the repository interface. The Firestore layer is drawn as the end state from the team's write-up (branch firebase).

2. Module dependency graph



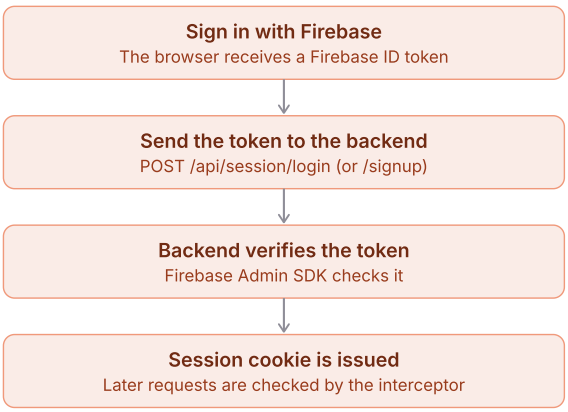
An arrow means “calls”. Connections, Groups and Matching call Student. Sessions calls Groups. Venues calls Sessions and Groups. Admin calls only Matching, for the config endpoints.

> All arrows are plain Java method calls inside one Spring Boot process (constructor injection), not network calls. REST only exists between the browser and the backend. All services also read and write through `StudyRepository`, which is why it is drawn once at the bottom.

> Admin is still the outlier: it does not use any other domain service. It reads and writes profiles, requests and groups straight through the repository inside one transaction, so deleting an account cleans up everything together.

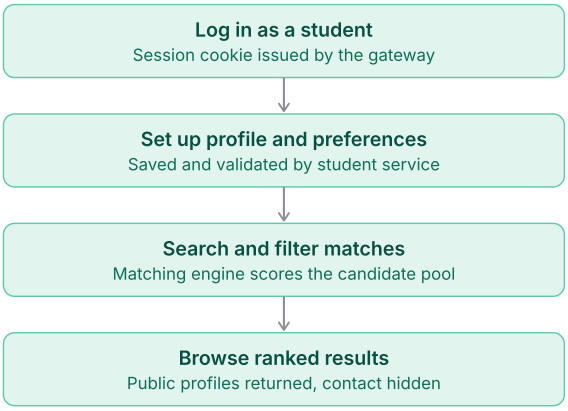
3. Process flows

Logging in

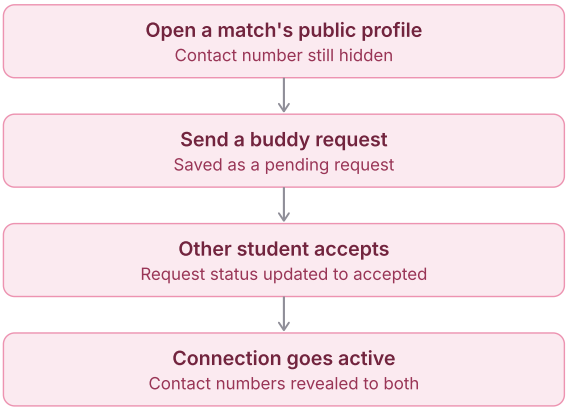


Students sign in through Firebase. The single administrator uses POST /api/session/admin-login with a username and password read from application.properties.

Finding a match



Becoming study buddies

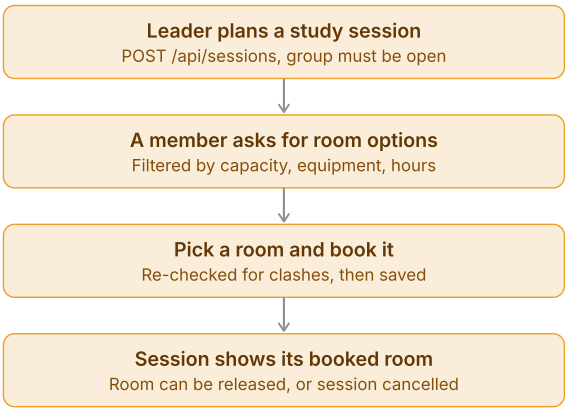


Forming a study group



Acceptance is re-checked when the leader decides, so two students can ask for the last seat and only one acceptance succeeds.

Planning a session and booking a room



Only the group leader (or an administrator) can plan or cancel a session. Any group member can request recommendations, book, or release a room.

4. Module and API reference

Session and login `/api/session`

Method	Path	Function
POST	`/api/session/login`	Verify a Firebase ID token and start a session
POST	`/api/session/signup`	Verify the token, create a student account, start a session
POST	`/api/session/admin-login`	Administrator username and password login
GET	`/api/session`	Current logged-in account
DELETE	`/api/session`	Log out

Unchanged gateway code; login now goes through Firebase.

Student `/api/students`

Method	Path	Function
GET	`/api/students`	List public profiles (contact hidden)
GET	`/api/students/courses`	Course catalogue
GET	`/api/students/{id}`	Viewer-aware public profile
POST	`/api/students`	Create or update profile, courses and preferences
GET	`/api/students/{id}/matches`	Ranked matches with filters for course, goal, mode, arrangement, day/time, overlap

Matching engine lives here (no separate controller). Depends on StudyRepository only.

Connections `/api/connections`

Method	Path	Function
POST	`/api/connections/requests`	Send a buddy request with an optional message
POST	`/api/connections/requests/{id}/status`	Accept or decline a pending request
GET	`/api/connections/students/{id}`	Request history (inbox and sent)
GET	`/api/connections/students/{id}/active`	Active connections

Depends on StudentService (validation, contact reveal) and StudyRepository.

Groups `/api/groups`

Method	Path	Function
GET	`/api/groups`	List groups
GET	`/api/groups/join-requests`	Join requests (leader view)
POST	`/api/groups`	Create a group
PUT	`/api/groups/{id}`	Update group details
POST	`/api/groups/{id}/join-requests`	Student asks to join
POST	`/api/groups/join-requests/{id}/decision`	Leader accepts or declines
POST	`/api/groups/{id}/remove-member`	Remove a member
POST	`/api/groups/{id}/transfer-leadership`	Transfer leadership
POST	`/api/groups/{id}/close`	Close the group
POST	`/api/groups/{id}/leader-quits`	Leader exit rule: close or transfer

Depends on StudentService and StudyRepository.

Sessions (new) `/api/sessions`

Method	Path	Function
POST	`/api/sessions`	Plan a study session for a group (leader)
GET	`/api/sessions?groupId=`	Sessions of one group, sorted by date
GET	`/api/sessions/mine`	Sessions across the caller's active groups
POST	`/api/sessions/{id}/cancel`	Cancel a session and its room booking (leader)

Depends on StudyGroupService and StudyRepository.

Venues (new) `/api/venues`

Method	Path	Function
GET	`/api/venues/rooms`	All rooms
GET	`/api/venues/recommendations?sessionId=`	Rooms that fit the session; optional building, whiteboard, projector, power filters
POST	`/api/venues/bookings?sessionId=&roomId=`	Book a room for a session
POST	`/api/venues/bookings/release?sessionId=`	Release the room booking

Depends on StudySessionService, StudyGroupService and StudyRepository.

Admin `/api/admin`

Method	Path	Function
GET	`/api/admin/accounts`	Accounts with usage (profile, connections, groups, pending)
POST	`/api/admin/accounts`	Create or update an account
DELETE	`/api/admin/accounts/{id}`	Delete an account and clean up related records
GET	`/api/admin/accounts/{id}/detail`	An account's buddies and groups (new)
GET	`/api/admin/matching-config`	Current matching weights and strategy
PUT	`/api/admin/matching-config`	Update matching weights and strategy

Depends on StudyRepository directly, plus MatchingService for config only.

5. Appendix

Firestore collections

Each type of data is a collection and each record is a document named by its id. From the team's Firestore write-up.

Collection	Holds
`UserAccount`	Login account, role, status
`StudentProfile`	Name, courses, availability, preferences
`Course`	Course code and name
`BuddyRequest`	Buddy request and its status
`StudyGroup`	Group details, leader, members
`MembershipRequest`	Request to join a group
`StudySession`	Scheduled study sessions
`Room / RoomBooking`	Venues and bookings
`MatchingProperties`	Admin matching weights and strategy (document id: current)
`SeedMarker`	Marks that demo data has been loaded
`_meta`	Lock document so business writes happen one at a time

How writes stay consistent

From the team's Firestore write-up. Service methods that change data are marked `@StudyTransactional`. The whole method runs as one Firestore transaction: saves and deletes are held back and written together at the end, and if the method throws an error nothing is written. `repository.lock()` uses the `_meta/lock` document so two writes at the same moment go one after the other.

What happens when the API is called (example: sending a buddy request)

1. **Student clicks the button in the browser.**
   React builds a JSON object like `{ senderId, recipientId, message }`.
2. **The browser sends an HTTP request.**
   A POST to `/api/connections/requests` with that JSON as the body.
3. **Spring Boot routes it.**
   `@PostMapping("/requests")` on `ConnectionController` maps the URL to a Java method.
4. **The backend does the work.**
   `ConnectionController` → `ConnectionService` (validates via `StudentService`) → `StudyRepository`.
5. **The backend responds.**
   Success, or an error such as “student not found”.
6. **React updates the screen.**
   The frontend never touches the database or business rules directly.

Moving boxes in Inkscape

1. **Select the whole box.**
   Rectangle and text are not grouped, so drag a selection rectangle around the box.
2. **Group it**
   with Ctrl+G (Cmd+G on Mac) so it moves as one object.
3. **Drag or nudge**
   with the mouse or arrow keys; Object > Transform (Shift+Ctrl+M) for exact values.
4. **Reconnect arrows by hand.**
   Lines are not attached to boxes; use the Node tool (N) to drag their endpoints.